

Entity Framework Core ve SQLite

Bellekteki ürün listesini gerçek bir veritabanına taşı. DbContext, DbSet ve migration kavramlarıyla kalıcı veri akışını kur.

Yaklaşık	Odak	Çıktı
180–240 dk	Veri kalıcılığı	products.db

Bugünün çalışma kuralı

Önce veritabanını oluştur ve doğrula. Service kodunu ancak migration başarıyla çalıştıktan sonra değiştir.

Bitiş ölçütü

- RAM ve veritabanı farkını açıklamak
- DbContext ile DbSet görevlerini ayırmak
- Migration oluşturup SQLite dosyasını üretmek
- POST ile eklenen ürünü yeniden başlatma sonrasında görmek

1. Bellek neden yeterli değil?

Gün 10'da ürünler ProductService içindeki List<Product> alanında tutuldu. Bu liste RAM'dedir ve uygulama kapanınca yok olur.

dotnet run

ProductService oluşur --> Liste RAM'e gelir

Ctrl+C --> RAM'deki liste kaybolur

RAM listesi	Veritabanı
Uygulama çalışırken yaşar	Diskte kalır
Yeniden başlatınca sıfırlanır	Yeniden başlatınca korunur
Öğrenme/test için kolay	Gerçek uygulama için uygundur

SQLite

Tek bir dosyada çalışan hafif bir ilişkisel veritabanıdır. Ayrı bir veritabanı sunucusu kurmadan EF Core öğrenmek için uygundur.

Bugünkü hedef

```

POST /api/products
    ↓
Entity Framework Core
    ↓
products.db
    ↓
Uygulama yeniden başlasa da ürün korunur

```

2. EF Core'un temel parçaları

Entity Framework Core, C# nesneleri ile ilişkisel veritabanı arasında köprü kuran bir ORM aracıdır.

Kavram	Görevi
Entity	Tablodaki bir satırı temsil eden C# class'ı
DbContext	Veritabanı bağlantısı ve çalışma alanı
DbSet	Bir entity tablosuna erişim yüzeyi
Provider	SQLite gibi kullanılacak veritabanı türü
Migration	Model değişikliklerini veritabanı şemasına taşır

Kavramsal eşleşme

C#	SQLite
-----	-----
Product class	Products tablosu
Product nesnesi	Products satırı
Product.Id	Id sütunu
Product.Name	Name sütunu
Product.Price	Price sütunu

ORM

Object-Relational Mapper: SQL tabloları ile C# object'leri arasında eşleme yapar. Her işlemde elle SQL yazma zorunluluğunu azaltır.

Akış

Controller --> Service --> DbContext --> SQLite

3. Proje ve paketler

Gün 10 projesinin yalnızca kaynak dosyalarını yeni Gün 11 projesine taşı. bin, obj ve eski csproj dosyasını kopyalama.

Proje oluşturma

```
cd "C:\Users\Administrator\.NetEgitim\DotNetIkinciHafta"
dotnet new webapi -n Gun11-EfCoreSqlite
```

Gerekli NuGet paketleri

```
cd Gun11-EfCoreSqlite
dotnet add package Microsoft.EntityFrameworkCore.Sqlite
dotnet add package Microsoft.EntityFrameworkCore.Design
```

EF komut aracı

```
dotnet tool update --global dotnet-ef

dotnet ef --version
```

Sürüm uyumu

Paket kurulumu uyumsuz sürüm hatası verirse SDK ve hedef framework sürümünü kontrol et; rastgele sürüm denemek yerine hata mesajını incele.

Önerilen yapı

```
Gun11-EfCoreSqlite/
├── Controllers/ProductsController.cs
├── Data/AppDbContext.cs
├── Dtos/CreateProductDto.cs
├── Models/Product.cs
├── Services/IProductService.cs
├── Services/ProductService.cs
├── Program.cs
└── appsettings.json
```

4. Entity ve DbContext

Product class'ı EF Core tarafından entity olarak kullanılır. Id property'si isimlendirme kuralı sayesinde primary key kabul edilir.

Models/Product.cs

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; } = "";
    public decimal Price { get; set; }
}
```

Data/AppDbContext.cs

```
using Microsoft.EntityFrameworkCore;

public class AppDbContext : DbContext
{
    public AppDbContext(
        DbContextOptions<AppDbContext> options)
        : base(options)
    {
    }

    public DbSet<Product> Products { get; set; }
    = null!;
}
```

Kod	Anlam
DbContext	EF Core'un veritabanı çalışma alanı
DbSet	Products tablosuna erişim
options	Bağlantı ve provider ayarları

null! neden var?

EF Core DbSet'i çalışma anında hazırlar. null! ifadesi derleyiciye bu property'nin framework tarafından atanacağını bildirir.

5. Bağlantıyı Program.cs'e eklemek

Önce EF Core namespace'ini ekle:

```
using Microsoft.EntityFrameworkCore;
```

DbContext kaydı

```
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlite(
        builder.Configuration.GetConnectionString(
            "DefaultConnection")));
```

Service yaşam süresi

```
builder.Services.AddScoped<
    IProductService,
    ProductService>();
```

appsettings.json

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Data Source=products.db"
  }
}
```

Ayar	Görevi
UseSqlite	SQLite provider'ını seçer.
DefaultConnection	Bağlantı ayarının ismidir.
Data Source	Oluşacak SQLite dosyasını gösterir.
AddScoped	Her HTTP isteğinde uygun service/context kapsamı sağlar.

Singleton neden kalktı?

DbContext varsayılan olarak Scoped çalışır. Aynı bağımlılık zincirindeki ProductService'i de Scoped kaydetmek güvenli ve uyumlu seçimdir.

6. Migration ve veritabanı

Migration, C# modelinden veritabanı tablosunu oluşturacak değişiklik planını üretir.

Migration oluştur

```
dotnet ef migrations add InitialCreate
```

Veritabanına uygula

```
dotnet ef database update
```

Oluşan dosyalar

```
Migrations/  
├── .._InitialCreate.cs  
├── .._InitialCreate.Designer.cs  
└── AppDbContextModelSnapshot.cs  
  
products.db
```

Model

`dotnet ef migrations add --> Migration`

`dotnet ef database update --> products.db`

Migration kaynak koddur

Migrations klasörü Git'e eklenir. products.db geliştirme tercihlerine göre Git dışında tutulabilir; gerçek projede veritabanı dosyasını paylaşmak çoğunlukla önerilmez.

Doğrulama

- Komutlar hatasız tamamlandı mı?
- Migrations klasörü oluştu mu?
- products.db dosyası oluştu mu?

7. Service'i veritabanına bağlamak

ProductService artık List<Product> tutmaz; ApplicationDbContext üzerinden Products tablosunu kullanır.

```
public class ProductService : IProductService
{
    private readonly ApplicationDbContext _context;

    public ProductService(ApplicationDbContext context)
    {
        _context = context;
    }

    public List<Product> GetAll()
    {
        return _context.Products.ToList();
    }

    public Product? GetById(int id)
    {
        return _context.Products.FirstOrDefault(
            product => product.Id == id);
    }

    public Product Create(CreateProductDto dto)
    {
        Product product = new Product
        {
            Name = dto.Name,
            Price = dto.Price
        };

        _context.Products.Add(product);
        _context.SaveChanges();

        return product;
    }
}
```

Kalıcılık testi

- POST ile ürün ekle ve 201 cevabını gör.
- GET ile ürünü listede doğrula.
- Ctrl+C ile API'yi durdurup yeniden çalıştır.
- GET sonucunda ürün hâlâ varsa veri kalıcıdır.

Git

```
git add .
git commit -m "Gün 11 EF Core ve SQLite tamamlandı"
git push
```

Gün 11 özeti

Controller HTTP'yi, service iş akışını, DbContext ise veritabanı erişimini yönetir. SQLite veriyi products.db içinde kalıcı tutar.